

Ejercicio paso a paso

En este ejercicio, crearemos un proyecto Maven más completo y modularizado. Organizaremos el código para que sea mantenible, escalable y robusto ante errores del usuario. Seguiremos trabajando con la misma base de datos haciendo las operaciones CRUD a través de un menú por consola

Estructura del Proyecto (Arquitectura DAO)

Para que el proyecto esté modularizado correctamente, seguiremos esta estructura de paquetes:

- com.escuela.model: Contiene la clase (POJO) que representa los datos.
- com.escuela.database: Gestión de la conexión.
- com.escuela.dao: La lógica de acceso a datos (Data Access Object).
- com.escuela.util: Utilidades como validación de entrada.
- com.escuela.main: El menú y la interacción con el usuario.

Paso 1: Configuración de Maven (pom.xml)

Crea un proyecto Maven y asegúrate de tener la dependencia del conector en el fichero pom.xml

```
<dependencies>
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <version>8.3.0</version>
  </dependency>
</dependencies>
```

Paso 2: El Modelo (Estudiante.java)

Es una clase simple que representa un objeto estudiante.

```
package com.escuela.model;

public class Estudiante {
    private int id;
    private String nombre;
    private String email;
    private double nota;

    // Constructor, Getters y Setters
    public Estudiante(int id, String nombre, String email, double nota) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
        this.nota = nota;
    }
}
```

Ejercicio paso a paso

```
@Override
public String toString() {
    return String.format("ID: %d | Nombre: %-15s | Email: %-20s | Nota: %.2f", id, nombre, email, nota);
}
// ... (Generar Getters y Setters)
}
```

Paso 3: Conexión Segura (ConexionBD.java)

Usaremos una clase dedicada para obtener la conexión. Esto centraliza la configuración.

```
package com.escuela.database;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {
    private static final String URL =
"jdbc:mysql://localhost:3306/escuela_db";
    private static final String USER = "root";
    private static final String PASS = "tu_password";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

Paso 4: Lógica de Datos (EstudianteDAO.java)

Aquí es donde usaremos los PreparedStatement. El resto del programa no sabe **cómo** se guardan los datos, solo llama a estos métodos.

```
package com.escuela.dao;

import com.escuela.database.ConexionBD;
import com.escuela.model.Estudiante;
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class EstudianteDAO {

    public List<Estudiante> listar() {
        List<Estudiante> lista = new ArrayList<>();
        String sql = "SELECT * FROM estudiantes";
        try (Connection con = ConexionBD.getConexion();
            PreparedStatement pstmt = con.prepareStatement(sql);
            ResultSet rs = pstmt.executeQuery()) {

            while (rs.next()) {
```

Ejercicio paso a paso

```
        lista.add(new Estudiante(rs.getInt("id"),
rs.getString("nombre"),
        rs.getString("email"), rs.getDouble("nota")));
    }
    } catch (SQLException e) { System.err.println("Error al listar: " +
e.getMessage()); }
    return lista;
}

public void insertar(String nombre, String email, double nota) {
    String sql = "INSERT INTO estudiantes (nombre, email, nota) VALUES
(?, ?, ?)";
    try (Connection con = ConexionBD.getConnection();
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, nombre);
        pstmt.setString(2, email);
        pstmt.setDouble(3, nota);
        pstmt.executeUpdate();
    } catch (SQLException e) { System.err.println("Error al insertar: " +
e.getMessage()); }
}

// ... Implementar métodos para eliminar(id) y actualizar(id, nota)
}
```

Paso 5: Validación de Entrada (InputReader.java)

Para evitar que el programa "explote" si el usuario mete una letra donde va un número, creamos esta utilidad.

```
package com.escuela.util;

import java.util.Scanner;

public class InputReader {
    private static Scanner sc = new Scanner(System.in);

    public static int leerEntero(String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                return Integer.parseInt(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Error: Por favor, introduce un número
entero válido.");
            }
        }
    }

    public static double leerDouble(String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                return Double.parseDouble(sc.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("Error: Introduce un número decimal (usa
el punto).");
            }
        }
    }
}
```

```
    }  
  }  
}  
  
public static String leerTexto(String mensaje) {  
    System.out.print(mensaje);  
    return sc.nextLine();  
}  
  
    //... poner métodos que sean necesarios  
}
```

Paso 6: La Capa de Vista (com.escuela.view)

Crea EstudianteView. Es la **única** clase que usa System.out y Scanner.

- **Método mostrarMenu():** Pinta las opciones.
- **Método formularioAlta():** Pide los datos y devuelve un objeto Estudiante.
- **Método listar(List<Estudiante> lista):** Recorre la lista y la imprime.

```
package com.escuela.view;  
  
import com.escuela.model.Estudiante;  
import com.escuela.util.Validador;  
import java.util.List;  
  
public class EstudianteView {  
  
    public int mostrarMenu() {  
        System.out.println("\n--- GESTIÓN DE ESTUDIANTES ---");  
        System.out.println("1. Listar estudiantes");  
        System.out.println("2. Registrar nuevo");  
        System.out.println("3. Actualizar nota");  
        System.out.println("4. Eliminar estudiante");  
        System.out.println("5. Salir");  
        return InputReader.leerEntero("Seleccione opción: ");  
    }  
  
    public void mostrarLista(List<Estudiante> estudiantes) {  
        if (estudiantes.isEmpty()) {  
            System.out.println("No hay registros en la base de datos.");  
        } else {  
            System.out.println("\n--- LISTADO ACTUAL ---");  
            // Formato de tabla simple para consola  
            System.out.printf("%-5s | %-20s | %-20s | %-5s\n", "ID",  
"Nombre", "Email", "Nota");  
            System.out.println("-----");  
            for (Estudiante e : estudiantes) {  
                System.out.printf("%-5d | %-20s | %-20s | %-5.2f\n",  
                    e.getId(), e.getNombre(), e.getEmail(), e.getNota());  
            }  
        }  
    }  
}
```

Ejercicio paso a paso

```
}

public Estudiante pedirDatosRegistro() {
    System.out.println("\n--- Nuevo Estudiante ---");
    String nombre = Validador.leerTexto("Nombre: ");
    String email = Validador.leerTexto("Email: ");
    double nota = Validador.leerDouble("Nota: ");
    // El ID se pone en 0 porque lo asignará MySQL automáticamente (Auto-increment)
    return new Estudiante(0, nombre, email, nota);
}

public int pedirId(String accion) {
    return Validador.leerEntero("Ingrese el ID del estudiante a " + accion + ": ");
}

public double pedirNuevaNota() {
    return Validador.leerDouble("Ingrese la nueva nota: ");
}

public void mensaje(String texto) {
    System.out.println(">> " + texto);
}
}
```

Paso 7: El Controlador (com.escuela.controller)

Crea EstudianteController. Es el director de orquesta.

1. Recibe la orden de la **Vista**.
2. Pide los datos al **DAO**.
3. Le dice a la **Vista** que muestre el resultado.

```
package com.escuela.controller;

import com.escuela.dao.EstudianteDAO;
import com.escuela.model.Estudiante;
import com.escuela.view.EstudianteView;
import java.util.List;

public class EstudianteController {
    private final EstudianteDAO dao;
    private final EstudianteView vista;

    public EstudianteController(EstudianteDAO dao, EstudianteView vista) {
        this.dao = dao;
        this.vista = vista;
    }

    public void iniciar() {
        int opcion;
        do {
            opcion = vista.mostrarMenu();
            ejecutarOpcion(opcion);
        } while (opcion != 0);
    }
}
```

Ejercicio paso a paso

```
    } while (opcion != 5);
}

private void ejecutarOpcion(int opcion) {
    switch (opcion) {
        case 1 -> {
            List<Estudiante> lista = dao.listar();
            vista.mostrarLista(lista);
        }
        case 2 -> {
            Estudiante nuevo = vista.pedirDatosRegistro();
            dao.insertar(nuevo);
            vista.mensaje("Estudiante registrado con éxito.");
        }
        case 3 -> {
            int id = vista.pedirId("actualizar");
            double nota = vista.pedirNuevaNota();
            dao.actualizarNota(id, nota);
            vista.mensaje("Nota actualizada.");
        }
        case 4 -> {
            int id = vista.pedirId("eliminar");
            dao.eliminar(id);
            vista.mensaje("Operación de borrado finalizada.");
        }
        case 5 -> vista.mensaje("Saliendo del sistema...");
        default -> vista.mensaje("Opción no válida.");
    }
}
}
```

Paso 8: El Punto de Entrada (App.java)

En un sistema modular, el main solo se encarga de realizar la "Inyección de Dependencias" manual: crea los objetos y los conecta.

```
package com.escuela.main;
import com.escuela.controller.EstudianteController;
import com.escuela.dao.EstudianteDAO;
import com.escuela.view.EstudianteView;

public class App {
    public static void main(String[] args) {
        // 1. Instanciar el acceso a datos (Modelo)
        EstudianteDAO dao = new EstudianteDAO();

        // 2. Instanciar la interfaz de usuario (Vista)
        EstudianteView vista = new EstudianteView();

        // 3. Crear el controlador pasándole sus herramientas (DAO y Vista)
        EstudianteController controller = new EstudianteController(dao,
vista);

        // 4. Arrancar la aplicación
        controller.iniciar();
    }
}
```